

NLP A3

Students : Subhasini Gupta (21AE3AI03) & Sachin Yadav (21CE3AI15)

4th April 2026

1 Task 2: Data Preprocessing

1.1 Dataset Loading and Filtering

The Fakeddit dataset was loaded from the directory `all_samples` (also includes `non_multimodal`), which contains three files: `all_train.tsv`, `all_test_public.tsv`, and `all_validate.tsv`. These files were read separately and then combined into a single unified dataset.

From the combined dataset, only the relevant columns were retained, namely `id`, `clean_title`, and `2_way_label`. Rows containing missing values were removed to ensure data quality. Additionally, the `clean_title` column was converted to string format and the labels were cast to integer type for consistency.

After preprocessing, the dataset size reduced slightly due to the removal of incomplete entries.

1.2 Dataset Statistics

The original combined dataset consisted of **1,063,106** posts. After filtering and cleaning, the dataset contained **971,806** valid samples.

The class distribution in the cleaned dataset is approximately balanced:

- Real (label 0): 487,085 samples
- Fake (label 1): 484,721 samples

1.3 Balancing the Dataset

Although the dataset is already nearly balanced, a smaller balanced subset was created to make training computationally efficient.

To achieve this, equal numbers of samples were randomly selected from both classes. The sampling size was determined as the minimum between the available class counts and a predefined limit of 20,000 samples per class. As a result:

- Fake samples selected: 20,000
- Real samples selected: 20,000

These samples were combined and shuffled to remove any ordering bias and ensure proper mixing of both classes.

The final balanced dataset contained **40,000** samples.

1.4 Train-Test Split

The balanced dataset was divided into training and testing sets using an 80-20 split. Stratified sampling was applied during the split to preserve the class distribution in both subsets.

The resulting dataset sizes are:

- Training set: 32,000 samples
- Testing set: 8,000 samples

The class distribution in both sets remains perfectly balanced:

Training Set:

- Real (0): 16,000
- Fake (1): 16,000

Testing Set:

- Real (0): 4,000
- Fake (1): 4,000

2 Task 1: Implementing BERT from Scratch

The model was implemented with an embedding dimension of 512, 8 attention heads, and 2 transformer encoder layers.

2.1 Model Architecture

The implementation consists of four main components: Multi-Head Self-Attention, Feed-Forward Network, Transformer Encoder Layer, and the overall BERT model.

The Multi-Head Self-Attention module projects the input embeddings into query, key, and value vectors, each of dimension 512. These are then split across 8 attention heads, resulting in a head dimension of 64. Attention scores are computed using scaled dot-product attention, followed by a softmax operation. The outputs from all heads are concatenated and passed through a final linear projection.

The Feed-Forward Network consists of two linear layers, expanding the embedding dimension from 512 to 2048 and then projecting it back to 512. A ReLU activation function is applied between the two layers.

Each Transformer Encoder layer combines these two components with residual connections and layer normalization. The input is first passed through the multi-head attention block, followed by normalization, and then through the feed-forward network with another normalization step.

The BERT model stacks two such encoder layers. It also includes token embeddings and positional embeddings, which are summed to form the input representation. A pooler layer extracts the representation of the [CLS] token, which is then passed through a classifier to produce output probabilities for binary classification.

2.2 Tokenization and Input Preparation

The `bert-base-uncased` tokenizer from Hugging Face was used to tokenize the input sentence:

The transformer model processes entire sequences in parallel and captures relationships between words effectively

The tokenizer splits the sentence into subword tokens and adds special tokens such as [CLS] and [SEP]. The sequence was padded to a fixed length of 20 tokens.

Tokenized Output:

- Tokens: [the, transform, ##er, model, processes, entire, sequences, in, parallel, and, captures, relationships, between, words, effectively]
- Tokens with special tokens: includes [CLS] at the beginning and [SEP] at the end, followed by padding tokens.

The corresponding input IDs and attention masks were generated to represent the tokenized sequence.

2.3 Embedding Stage

The input IDs were first passed through the token embedding layer and positional embedding layer. Both embeddings have the same dimensionality and are added element-wise.

- Token Embeddings: (1, 20, 512)
- Positional Embeddings: (1, 20, 512)

The resulting tensor represents the input to the encoder stack.

2.4 Encoder Layers

The model contains two Transformer Encoder layers. The shape of the tensor remains consistent throughout the layers, which is a key property of transformer architectures.

Encoder Layer 1:

- Multi-Head Attention Output: $(1, 20, 512)$
- Feed-Forward Output: $(1, 20, 512)$
- Encoder Output: $(1, 20, 512)$

Encoder Layer 2:

- Multi-Head Attention Output: $(1, 20, 512)$
- Feed-Forward Output: $(1, 20, 512)$
- Encoder Output: $(1, 20, 512)$

This consistency in dimensions ensures that residual connections can be applied without shape mismatch.

2.5 Classification Head

After passing through the encoder layers, the representation corresponding to the [CLS] token (first token) is extracted. This vector has shape $(1, 512)$ and serves as a summary of the entire sequence.

The pooled output is then passed through a linear classifier followed by a softmax function to obtain class probabilities:

- Classifier Output: $(1, 2)$

Final Output Probabilities:

$[0.5339, 0.4661]$

2.6 Parameter Shapes

The model parameters were inspected to verify the correctness of implementation. The embedding layers, attention projections, feed-forward layers, normalization layers, and classifier all have expected dimensions.

For example:

- Token Embedding Matrix: $(30522, 512)$
- Position Embedding Matrix: $(32, 512)$
- Attention Projection Matrices: $(512, 512)$
- Feed-Forward Layers: $(512 \rightarrow 2048 \rightarrow 512)$
- Classifier: $(512 \rightarrow 2)$

3 Task 3: Fine-Tuning BERT and Reporting Metrics

3.1 Model Selection

For this task, the pre-trained `bert-base-uncased` model from Hugging Face was used. The model was instantiated using `BertForSequenceClassification`, which adds a classification head on top of the BERT encoder for binary classification (fake vs real news).

Instead of fully fine-tuning all parameters, a partial fine-tuning strategy was adopted. The embedding layer and the first six encoder layers were frozen, while the remaining layers were kept trainable. This reduces computational cost and helps prevent overfitting while still allowing the model to adapt to the task.

3.2 Data Preparation

The dataset prepared in the previous task was used for training. The training data was further split into training and validation subsets using stratified sampling to preserve class distribution.

The resulting split sizes are as follows:

- Training set: 27,200 samples
- Validation set: 4,800 samples
- Test set: 8,000 samples

Stratification ensures that both classes are equally represented across all splits, which is critical for stable training in binary classification tasks.

3.3 Tokenization and Dataset Construction

The input text (cleaned news titles) was tokenized using the `bert-base-uncased` tokenizer. Each text sample was converted into input IDs and attention masks.

A custom PyTorch `Dataset` class was implemented to handle tokenization dynamically. The following configuration was used:

- Maximum sequence length: 64
- Padding: `max_length`
- Truncation: enabled

Each sample returns `input_ids`, `attention_mask`, and the corresponding label, making it directly compatible with the Hugging Face training pipeline.

3.4 Training Configuration

Training was performed using mini-batch gradient descent with a batch size of 16. The AdamW optimizer was used with weight decay regularization, applied only to trainable parameters.

The key hyperparameters are:

- Epochs: 5
- Batch size: 16
- Learning rate: 1×10^{-5}
- Optimizer: AdamW (with weight decay = 0.01)
- Gradient clipping: 1.0

The model contains a total of 109,483,778 parameters, out of which 43,119,362 are trainable due to layer freezing.

3.5 Training Strategy

The model was trained using a standard supervised learning pipeline with back-propagation and gradient-based optimization. During training, both training loss and validation loss were monitored at the end of each epoch.

To improve generalization and prevent overfitting, early stopping was implemented with a patience of 2 epochs. The model with the lowest validation loss was saved.

The training process is summarized below:

Epoch	Train Loss	Validation Loss
1	0.4974	0.4112
2	0.3775	0.3808
3	0.3282	0.3923
4	0.2949	0.3949

The validation loss decreased initially and reached its minimum at epoch 2. After that, it started increasing, indicating the onset of overfitting. Early stopping was triggered at epoch 4.

3.6 Evaluation Metrics

After training, the best-performing model (based on validation loss) was loaded and evaluated on the test set. The following standard classification metrics were computed:

- Accuracy
- Precision
- Recall
- F1 Score

3.7 Results

The model achieved the following performance on the test set:

Metric	Value
Accuracy	0.8346
Precision	0.8396
Recall	0.8273
F1 Score	0.8334

3.8 Confusion Matrix

$$\begin{bmatrix} 3368 & 632 \\ 691 & 3309 \end{bmatrix}$$

3.9 Interpretation of Results

The model achieved an accuracy of approximately 83.5%, which indicates strong performance for the fake news classification task. Compared to the previous experiment performed (in which I did full finetuning), the performance improved, likely due to better regularization and controlled fine-tuning.

Precision and recall are well balanced, suggesting that the model does not heavily favor one class over the other. The slightly higher precision indicates that false positives are relatively well controlled, while recall shows that most relevant instances are correctly identified.

The confusion matrix further confirms that both false positives and false negatives are reasonably low relative to the dataset size, indicating stable and reliable predictions.

4 Task 4: Model Explainability using Integrated Gradients

4.1 Objective

In this task, the predictions of the fine-tuned BERT model were interpreted using Integrated Gradients (IG). The goal was to identify which input tokens contributed the most to the model’s classification decision for both fake and real news samples.

4.2 Model Loading

The fine-tuned model obtained from Task 3 was saved and reloaded using the Hugging Face interface. The model was set to evaluation mode to disable dropout and ensure deterministic behavior during attribution computation.

4.3 Input Selection

To analyze model behavior, a total of six samples were selected from the test dataset, consisting of three fake and three real examples. Stratified sampling was used to ensure balanced representation.

The selected sentences are shown below:

1. (Fake) look at the black wonder what it was
2. (Fake) this still has the pressure inside from airplane altitude
3. (Fake) friezas graduation album
4. (Real) eww what the hell
5. (Real) foreboding panic total control of world fisting
6. (Real) spider laying eggs on human face

Each sentence was tokenized using the same tokenizer configuration as in Task 3, generating `input_ids`, `attention_mask`, and `token_type_ids`. When token type IDs were not available, zero tensors were used as a fallback.

4.4 Integrated Gradients Setup

Integrated Gradients (IG) was implemented using the Captum library. Since BERT operates on embeddings rather than raw token IDs, attributions were computed at the embedding layer using `LayerIntegratedGradients`.

A custom forward function was defined that takes input embeddings along with attention masks and token type IDs, and returns the model logits. This enables gradient computation with respect to the input embeddings.

A zero embedding tensor of the same shape as the input was used as the baseline. This baseline represents an absence of information, allowing IG to measure the contribution of each token relative to a neutral input. For each sentence, attributions were computed with respect to the predicted class index.

A key aspect of IG is the choice of the number of integration steps (`n_steps`), which controls the approximation quality of the path integral. Instead of using a fixed value, `n_steps` was tuned adaptively for each input. Specifically, values in the range $\{50, 100, 150, 200, 250, 300\}$ were evaluated, and the convergence delta returned by IG was used as a measure of approximation error.

The selection strategy was as follows:

- The smallest `n_steps` achieving a convergence delta below 0.01 was selected.
- If no such value was found, the configuration with the lowest absolute delta was chosen.

This adaptive strategy ensures a balance between computational efficiency and attribution accuracy.

4.5 Attribution Processing

The attribution tensor produced by IG has the same dimensionality as the embeddings. To obtain a single importance score per token, attribution values were summed across the embedding dimension. The resulting scores were normalized using the ℓ_2 norm to improve interpretability.

Token IDs were converted back to tokens using the tokenizer, and special tokens such as [CLS], [SEP], and [PAD] were removed from the analysis. The tokens were then ranked based on the absolute magnitude of their attribution scores.

4.6 Results

For each input sentence, the top-5 most important tokens are reported along with the selected number of integration steps and the final convergence delta.

Sentence 1 (Fake): look at the black wonder what it was

Selected `n_steps`: 100, Convergence delta: -0.0043

- what (0.2803)
- wonder (0.1648)
- was (-0.1043)
- it (-0.0859)
- black (-0.0617)

Sentence 2 (Fake): this still has the pressure inside from airplane altitude

Selected `n_steps`: 250, Convergence delta: -0.0281

- this (0.5889)
- altitude (0.3060)
- has (0.1907)
- airplane (0.1145)
- inside (0.0949)

Sentence 3 (Fake): friezas graduation album

Selected **n_steps**: 200, Convergence delta: 0.0037

- album (-0.3207)
- s (-0.1509)
- ie (-0.0477)
- za (-0.0468)
- fr (-0.0328)

Sentence 4 (Real): eww what the hell

Selected **n_steps**: 300, Convergence delta: -0.0293

- what (0.1966)
- hell (0.1253)
- e (-0.0557)
- w (0.0508)
- w (-0.0155)

Sentence 5 (Real): foreboding panic total control of world fisting

Selected **n_steps**: 300, Convergence delta: 0.0081

- fist (-0.1700)
- panic (0.0693)
- bo (-0.0679)
- control (0.0658)
- total (-0.0516)

Sentence 6 (Real): spider laying eggs on human face

Selected **n_steps**: 300, Convergence delta: -0.0048

- eggs (0.4996)
- laying (0.3664)
- face (-0.2150)
- human (0.1713)
- spider (0.1643)

4.7 Conclusion

The results show that the model relies heavily on semantically meaningful tokens when making predictions. Words such as *what*, *wonder*, *altitude*, and *eggs* receive high attribution scores, indicating their strong influence on classification decisions.

In several cases, subword tokens (e.g., **##s**, **##ie**) also contribute to the prediction, highlighting how BERT processes words at the subword level. Negative attribution values indicate tokens that push the prediction away from the predicted class.

The convergence delta values are generally small (close to zero), which confirms that the IG approximation is reliable. Better Delta Values required the importance of tuning **n_steps**, i.e., the number of integration steps.